



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

Программной инженерии и компьютерной техники (ФПИ и КТ)

ОТЧЕТ

по лабораторной работе №1

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация алгоритмов хэширования (perfect, extendible, min)»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	Perfect hashing	3
1.2	Extendible hashing	3
1.3	Min hash	4
2	Практическая часть	5
2.1	Реализация perfect hash таблицы	5
2.2	Реализация extendible hash таблицы	5
2.3	Реализация min hash таблицы LSH индекса	6
3	Исследовательская часть	7
3.1	Аппаратные характеристики	7
3.2	Методика замеров perfect hashing	7
3.3	Методика замеров extendible hashing	7
3.4	Методика замеров min hash	8
3.5	Результаты для perfect hashing	8
3.6	Результаты для extendible hashing	10
3.7	Результаты для min hash	13
4	Вывод	17

1 Теоретическая часть

1.1 Perfect hashing

Perfect hash для множества ключей S — это хэш-функция h , которая отображает ключи из S в индексы таблицы без коллизий. В классическом варианте (FKS) используется двухуровневая схема: первичная таблица распределяет ключи по корзинам, а для каждой корзины строится вторичная таблица, в которой подбираются параметры так, чтобы коллизий не было. Основными идеями алгоритма являются:

- первичная таблица имеет размер, равный ближайшему простому $\geq n$;
- в ячейке хранится либо одна пара **key/value**, либо указатель на вторичную таблицу;
- при коллизии создаётся/перестраивается вторичная таблица: увеличивается её размер и перебираются различные хэш-функции до получения раскладки без коллизий.

1.2 Extendible hashing

Extendible hashing предназначен для динамических наборов данных, где число ключей заранее неизвестно и структура должна расти и уменьшаться без полной перестройки таблицы. В отличие от *perfect hashing*, здесь допускаются коллизии, но они локализуются в бакетах фиксированной ёмкости. Основные элементы алгоритма:

- директория содержит 2^d указателей на бакеты, где d — глобальная глубина;
- каждый бакет имеет локальную глубину, определяющую, по скольким младшим битам хэша он адресуется;
- при переполнении бакет расщепляется, а при необходимости директория удваивается;
- при удалении возможны обратные операции: слияние соседних бакетов и уменьшение директории.

Такая схема обеспечивает амортизированно быстрые операции `insert/get/update/delete` и хорошо подходит для внешней памяти, так как бакеты можно размещать непосредственно в файле и адресовывать через отображение файла в память.

1.3 Min hash

Min hash предназначен для эффективного поиска похожих объектов в больших коллекциях, используя хэширование, чувствительное к локальности. Мера сходства — коэффициент Жаккара $J(A, B) = |A \cap B| / |A \cup B|$, где A и B — множества *шинглов* (подстрок длины k , извлечённых скользящим окном) двух документов. *Min hash* даёт компактную аппроксимацию Жаккара, то есть для набора хэш-функций $h_1, \dots, h_m$ сигнатура документа — это вектор минимумов $\sigma_i = \min_{s \in A} h_i(s)$. Вероятность совпадения $\sigma_i(A) = \sigma_i(B)$ равна $J(A, B)$, поэтому долю совпавших компонент можно использовать как оценку сходства.

Locality-Sensitive Hashing (LSH) ускоряет поиск кандидатов — сигнатура делится на b полос по r элементов каждая. Два документа попадают в один бакет хотя бы одной полосы с вероятностью $1 - (1 - s^r)^b$, где s — истинный Жаккар. Порог чувствительности $s^* \approx (1/b)^{1/r}$ задаётся выбором b и r . Полосовая схема позволяет за линейное время построить список пар-кандидатов, а затем верифицировать их точным сравнением шинглов.

2 Практическая часть

2.1 Реализация perfect hash таблицы

Публичный интерфейс реализован в файлах `inc/perfect.h` и `src/perfect.c`:

- `create_perfect_table` создаёт таблицу нужного размера;
- `perfect_set` добавляет/обновляет пару ключ–значение;
- `perfect_get` выполняет поиск значения по ключу;
- `free_perfect_table` освобождает память.

Структура `perfect_hash_table_t` содержит массив указателей на `ceil_value_t`. Каждая ячейка либо хранит одну пару `key/value`, либо указывает на вторичную таблицу (вложенный `perfect_hash_table_t`). Для хэширования строк используются функции из `inc/hash.h`. При перестроении вторичных таблиц перебирается набор хэш–функций до построения раскладки без коллизий.

2.2 Реализация extendible hash таблицы

Основной код расположен в файле `extendible/extendible.go`. Таблица хранится не в оперативной памяти как отдельная структура с последующей сериализацией, а непосредственно в файле, отображённом в память с помощью `mmap`. Файл разбит на три логические области:

- 1) заголовок с метаданными таблицы (магическое число, версия формата, глобальная глубина, размер бакета);
- 2) директория фиксированного максимального размера, содержащая идентификаторы бакетов;
- 3) область бакетов, где для каждого бакета хранятся локальная глубина, число записей и массив пар `key/value/hash`.

Публичный интерфейс включает операции `Insert`, `Update`, `Delete`, `Get`, а также открытие и закрытие таблицы в файле. При переполнении вызывается `split` бакета, при удалении — `merge buddy`-бакетов и, если возможно, `shrink` директории.

2.3 Реализация min hash таблицы LSH индекса

Основные файлы: `min/minhash.go` (ядро алгоритма) и `min/workload.go` (генератор синтетического корпуса). Архитектура индекса:

- 1) из текста документа извлекаются шинглы функцией `shingleHashes` скользящим окном длины $k = 2$ слова; значение каждого шингла — FNV-1a хэш строки;
- 2) функцией `signatureFor` строится min hash-сигнатура из $m = 64$ хэш-функций на основе `splitmix64`;
- 3) при построении индекса (`Build`) сигнатуры всех документов разбиваются на $b = 8$ полос по $r = 8$ компонент; каждая полоса хэшируется FNV-полиномом в ключ бакета (`bandKey`);
- 4) при добавлении нового документа (`Add`) его бакеты объединяются с существующими;
- 5) поиск дубликатов (`FindDuplicates`) извлекает кандидатов из бакетов и проверяет пары точным подсчётом Жаккара на сохранённых шинглах;
- 6) `FullScanDuplicates` является базовым алгоритмом сравнения: каждый документ проверяется против всех остальных за $O(n^2)$.

Параметры m , b , r , k передаются через структуру `Config`.

3 Исследовательская часть

3.1 Аппаратные характеристики

Все замеры проводились на ноутбуке со следующей конфигурацией:

- операционная система: macOS 26.3.1 (arm64);
- процессор: Apple M3 Pro, 11 ядер;
- оперативная память: 18 ГБ;
- версия Go (для `extendible` и `min`): `go1.25`, версия C (для `perfect`): `c17`;

Во время замеров не выполнялись ресурсоёмкие фоновые задачи, а ноутбук был подключён к сети электропитания для обеспечения максимальной производительности.

3.2 Методика замеров `perfect hashing`

Для замеров использована программа `src/benchmark.c`. Для каждого датасета из `data/bench/manifest.txt`:

- 1) создаётся пустая `perfect hash` таблица;
- 2) измеряется время вставки всех n ключей (`perfect_set`);
- 3) измеряется время чтения всех n ключей (`perfect_get`).

Каждый прогон усредняется по `BENCH_ITERS` (по умолчанию 5).

3.3 Методика замеров `extendible hashing`

Для `extendible hashing` использованы встроенные `benchmark`-тесты Go и `pprof`. Измерялись четыре операции: `insert`, `update`, `delete`, `get`. Для каждой мощности набора данных выполнялись бенчмарки со считыванием времени на операцию, пропускной способности, количества аллокаций и объёма выделенной памяти.

3.4 Методика замеров min hash

Для min hash использованы встроенные benchmark-тесты Go и pprof. Измерялись три операции:

- **build** — построение индекса из корпуса размером N ;
- **add** — добавление N документов по одному в уже существующий индекс;
- **fullscan** — полный обход: для каждого из N документов запускается линейный поиск дубликатов по всему корпусу.

Для каждой мощности набора данных выполнялись бенчмарки со считыванием времени на операцию, пропускной способности, количества аллокаций и объёма выделенной памяти.

3.5 Результаты для perfect hashing

Таблица 3.1 содержит усреднённые значения задержек и пропускной способности.

Таблица 3.1 – Результаты бенчмарка perfect hash (среднее по итерациям)

N	вставка, нс/оп	вставка, млн оп/с	поиск, нс/оп	поиск, млн оп/с
50	344.0	2.98	80.0	12.50
250	528.0	1.90	112.0	8.95
1250	226.4	4.42	93.0	10.76
7500	314.9	3.24	82.8	12.19
20000	404.5	2.51	66.2	15.22
50000	217.0	4.61	52.4	19.09
100000	127.7	7.83	43.2	23.15
250000	225.7	4.43	58.3	17.14
500000	141.3	7.08	52.2	19.15
750000	138.7	7.21	52.0	19.24
1000000	197.4	5.07	57.9	17.29
2000000	164.9	6.07	57.2	17.51
5000000	194.1	5.15	81.2	12.32

Графики результатов замеров представлены на рисунках 3.1 и 3.2:

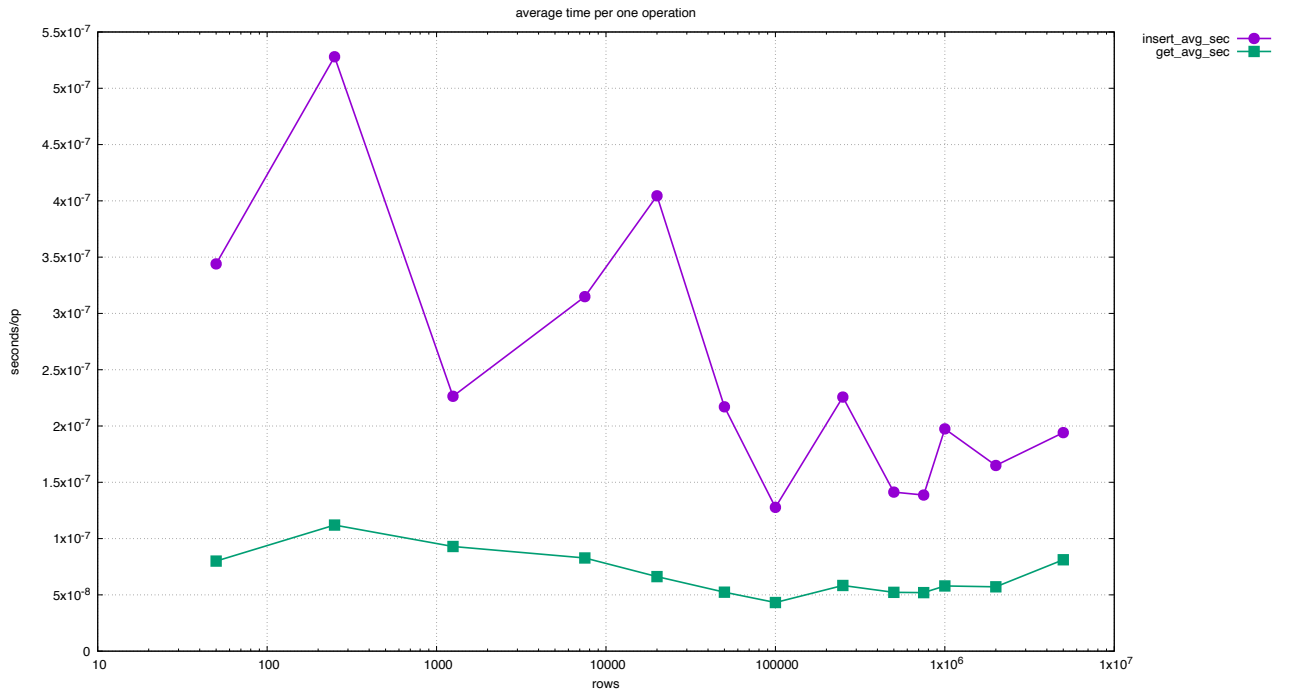


Рисунок 3.1 – Средняя задержка операций вставки/поиска для perfect hash

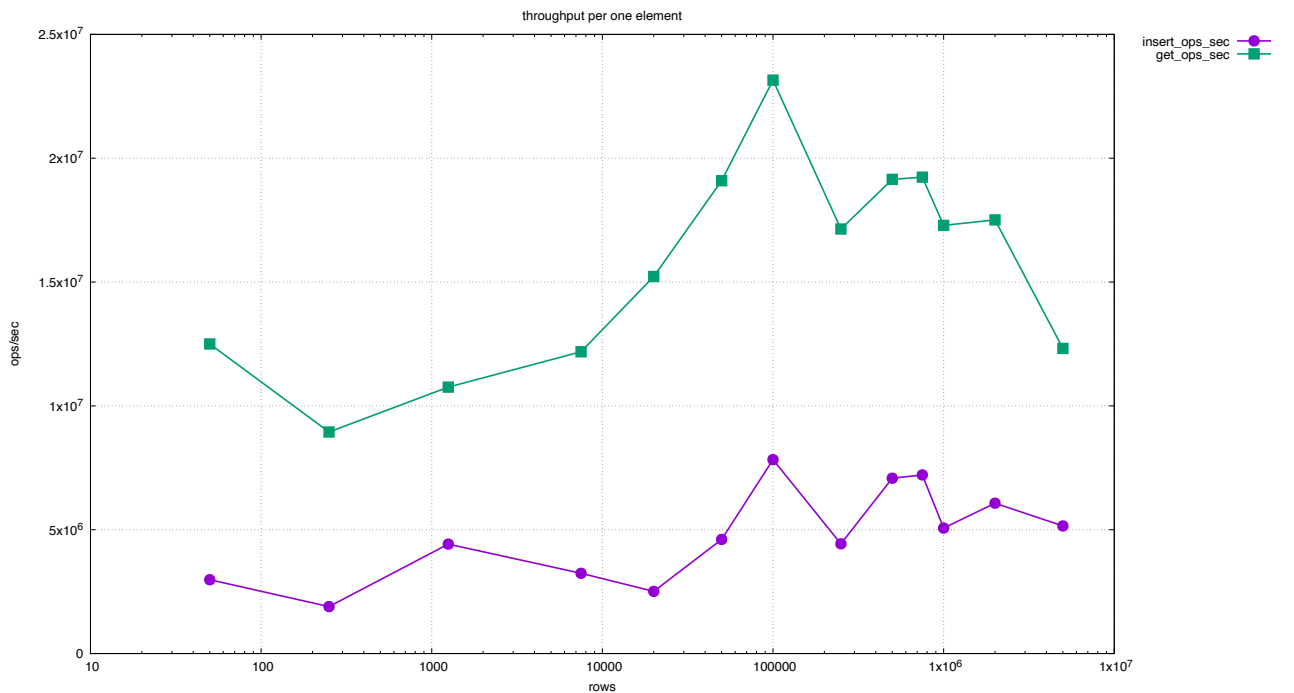


Рисунок 3.2 – Пропускная способность операций вставки/поиска для perfect hash

По таблице 3.1 и графикам можно сделать вывод, что операция `perfect_get` остаётся стабильной на всем диапазоне N и в среднем быстрее операции вставки. Операция `perfect_set` демонстрирует большую вариативность, так как стоимость зависит от распределения ключей по вторичным

таблицам и частоты их перестроений. В целом для статического или слабо изменяемого множества ключей perfect hashing обеспечивает высокую пропускную способность и предсказуемо малую задержку поиска.

3.6 Результаты для extendible hashing

Таблицы 3.2, 3.3 и 3.4 содержат результаты бенчмарка extendible hashing для всех измеренных размеров набора данных.

Таблица 3.2 – Результаты бенчмарка задержки в extendible hashing (среднее по итерациям)

N	итерации	вставка, нс/оп	обновление, нс/оп	удаление, нс/оп	поиск, нс/оп
1000	27894	43.42 ± 0.035	32.61 ± 0.019	40.86 ± 0.024	32.41 ± 0.020
5000	5374	44.61 ± 0.060	25.88 ± 0.023	35.37 ± 0.032	25.16 ± 0.023
10000	2610	45.85 ± 0.072	27.47 ± 0.028	40.36 ± 0.048	26.73 ± 0.028
50000	483	49.59 ± 0.109	30.86 ± 0.053	70.55 ± 0.355	30.81 ± 0.046
100000	208	57.37 ± 0.133	32.03 ± 0.063	94.91 ± 0.861	31.87 ± 0.051
500000	19	123.7 ± 0.699	36.31 ± 0.103	310.8 ± 6.781	35.95 ± 0.087
1000000	10	214.1 ± 2.413	54.02 ± 0.440	559.7 ± 19.71	53.44 ± 0.416

Таблица 3.3 – Результаты бенчмарка пропускной способности в extendible hashing (среднее по итерациям)

N	итерации	вставка, млн оп/с	обновление, млн оп/с	удаление, млн оп/с	поиск, млн оп/с
1000	27894	23.0 ± 0.018	30.7 ± 0.018	24.5 ± 0.015	30.9 ± 0.019
5000	5374	22.4 ± 0.030	38.6 ± 0.021	28.3 ± 0.025	39.7 ± 0.036
10000	2610	21.8 ± 0.034	36.4 ± 0.029	24.8 ± 0.047	37.4 ± 0.035
50000	483	20.2 ± 0.044	32.4 ± 0.053	14.2 ± 0.071	32.5 ± 0.047
100000	208	17.4 ± 0.051	31.2 ± 0.061	10.5 ± 0.097	31.4 ± 0.051
500000	19	8.1 ± 0.069	27.5 ± 0.098	3.2 ± 0.035	27.8 ± 0.083
1000000	10	4.7 ± 0.048	18.5 ± 0.151	1.8 ± 0.063	18.7 ± 0.146

Таблица 3.4 – Результаты замера времени прогрева (warmup) в extendible hashing

N	итерации	время warmup, нс/оп
1000	1681	815416 ± 128.5
5000	8356	1093872 ± 72.15
10000	1124	1090091 ± 196.1
50000	1166	1062745 ± 170.0
100000	283	3677399 ± 588.4
500000	250	6045153 ± 653.3
1000000	100	10683703 ± 1008

Из таблиц можно сделать вывод, что операции **get** и **update** наиболее стабильны по мере роста набора данных. Операция **insert** заметно дороже из-за split бакетов и расширения директории, а **delete** оказывается самой тяжёлой на больших N из-за merge/shrink после удаления элементов. Дополнительные профилировочные замеры показывают, что основной объём аллокаций приходится на **insert**; для **update**, **delete** и **get** объём дополнительных аллокаций почти не растёт и определяется в основном накладными расходами рантайма.

Графики результатов замеров extendible hashing представлены на рисунках ниже.

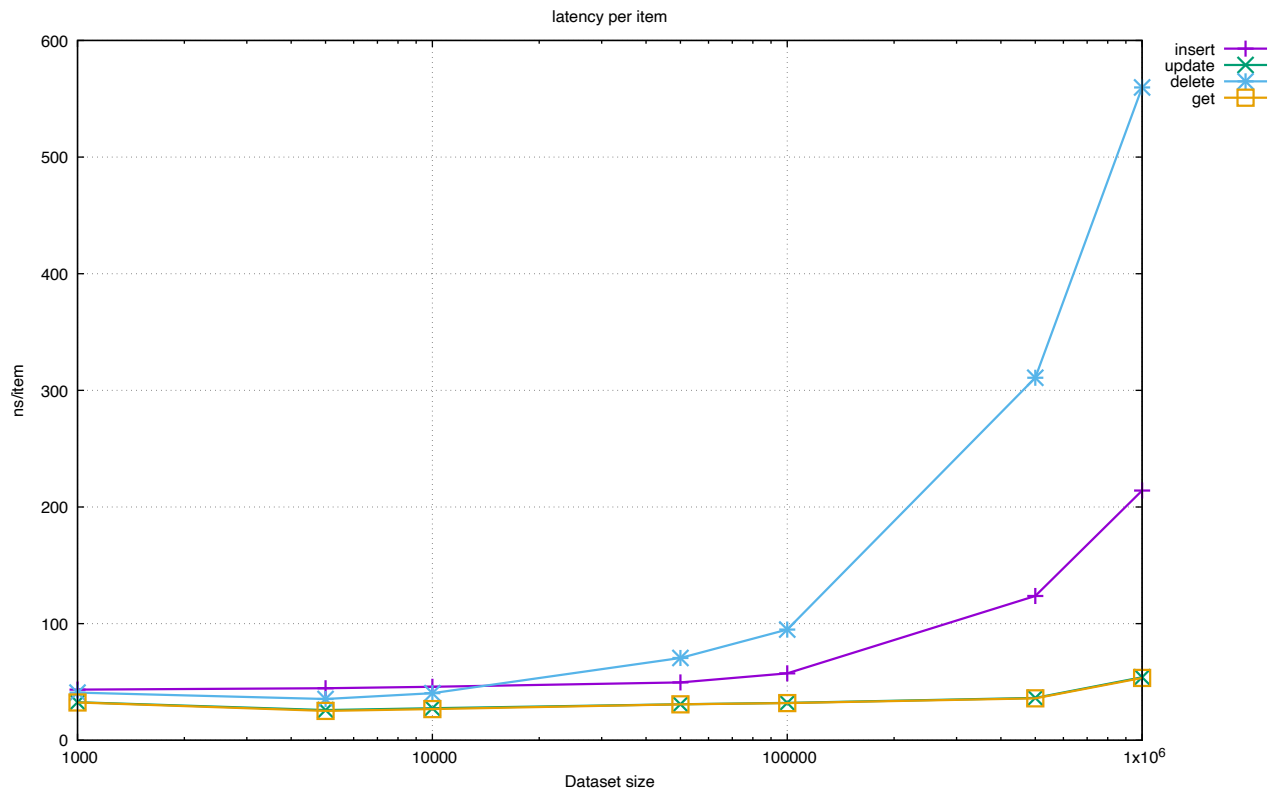


Рисунок 3.3 – Задержка операций extendible hashing на элемент

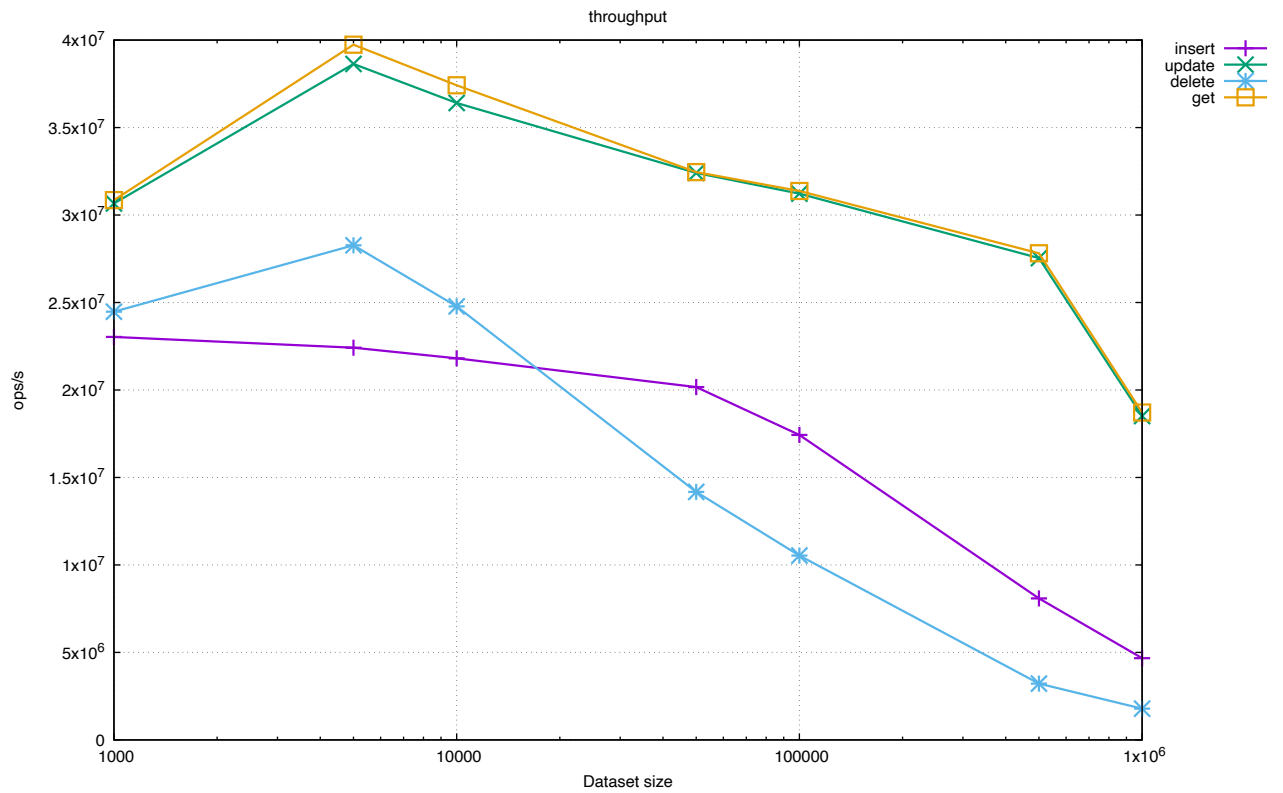


Рисунок 3.4 – Пропускная способность операций extendible hashing

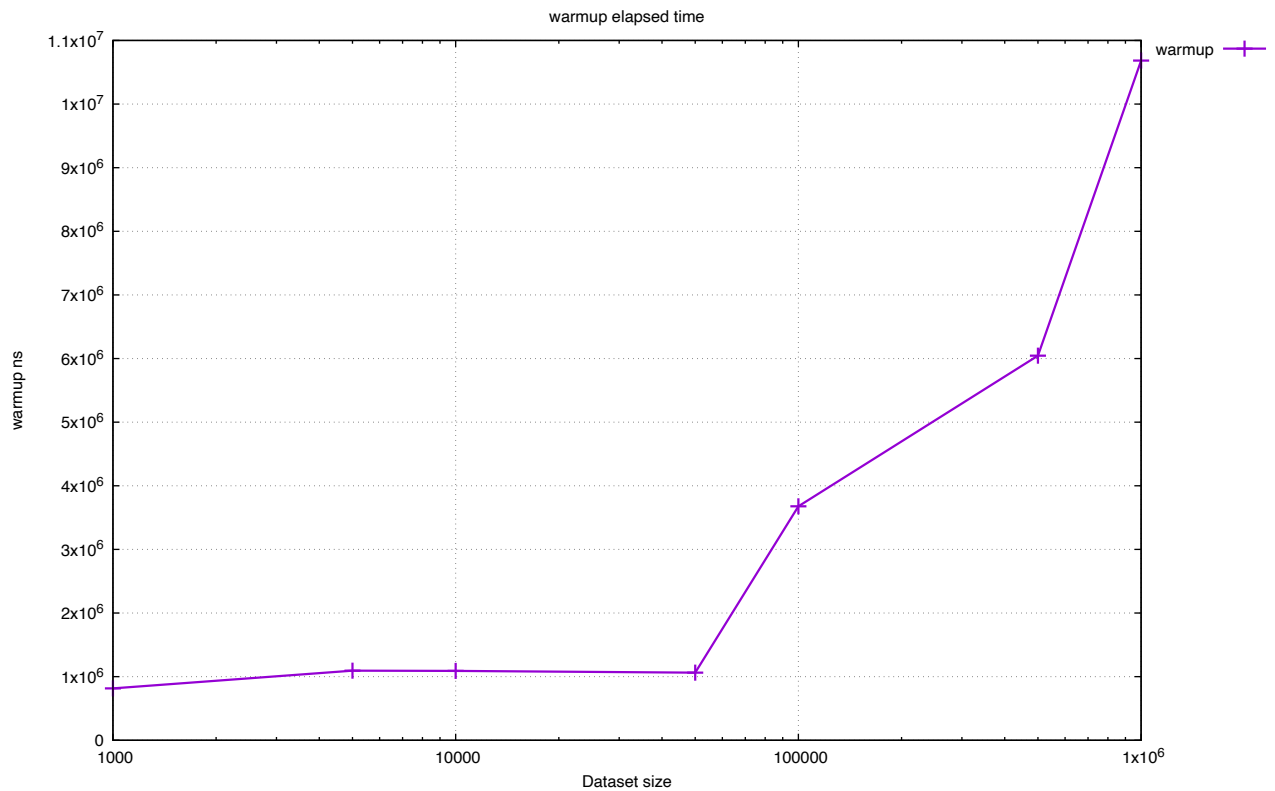


Рисунок 3.5 – Время прогрева (warmup) для extendible hashing

3.7 Результаты для min hash

Таблица 3.5 содержит задержку операций, таблица 3.6 — пропускную способность.

Таблица 3.5 – Результаты бенчмарк задержки в min hash (среднее по итерациям)

N	построение, нс/элемент	добавление, нс/элемент	полный обход, нс/элемент
500	15 941	16 307	278 298
1000	15 889	16 750	560 994
5000	16 126	16 949	2 798 846
10000	16 275	16 759	5 598 740

Таблица 3.6 – Результаты бенчмарка пропускной способности в min hash (среднее по итерациям)

N	построение, оп/с	добавление, оп/с	полный обход, оп/с
500	62 731	61 325	3593
1000	62 938	59 702	1783
5000	62 013	58 999	357
10000	61 444	59 668	179

Операции **build** и **add** демонстрируют практически постоянную задержку (≈ 16 тыс нс/элемент) при росте корпуса — это следствие того, что вычисление min hash-сигнатуры обрабатывает каждый документ независимо. Задержка **fullscan** растёт квадратично ($O(n^2)$), поскольку для каждого из N документов производится линейный перебор всего корпуса.

Графики результатов замеров min hash представлены на рисунках ниже.

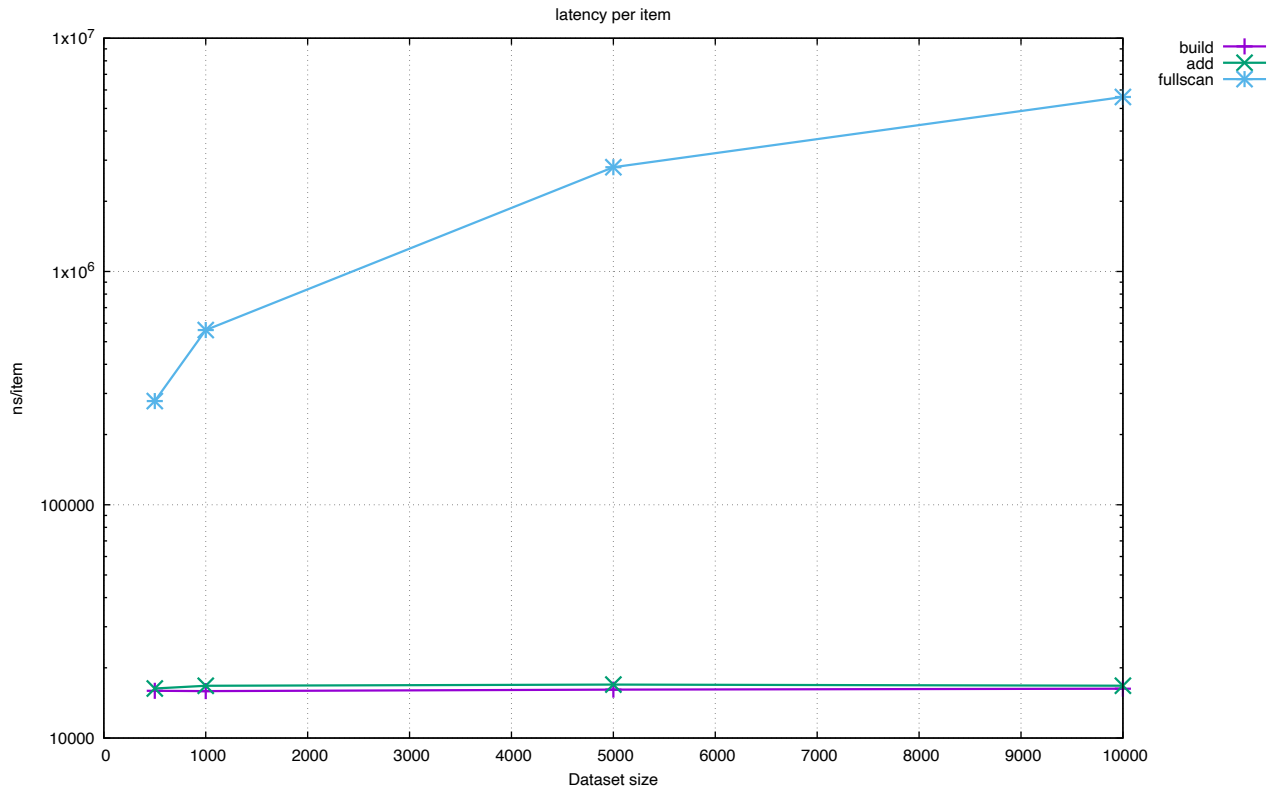


Рисунок 3.6 – Задержка операций min hash на элемент

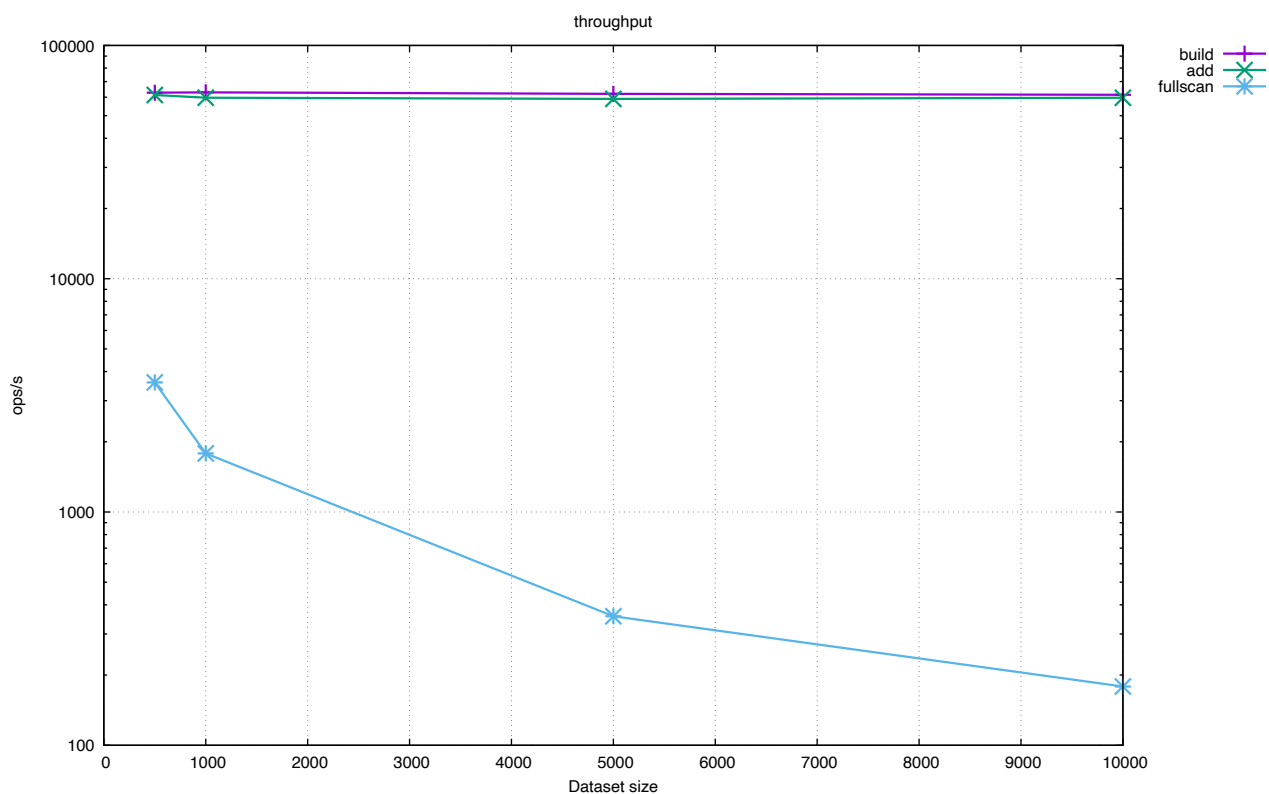


Рисунок 3.7 – Пропускная способность операций min hash

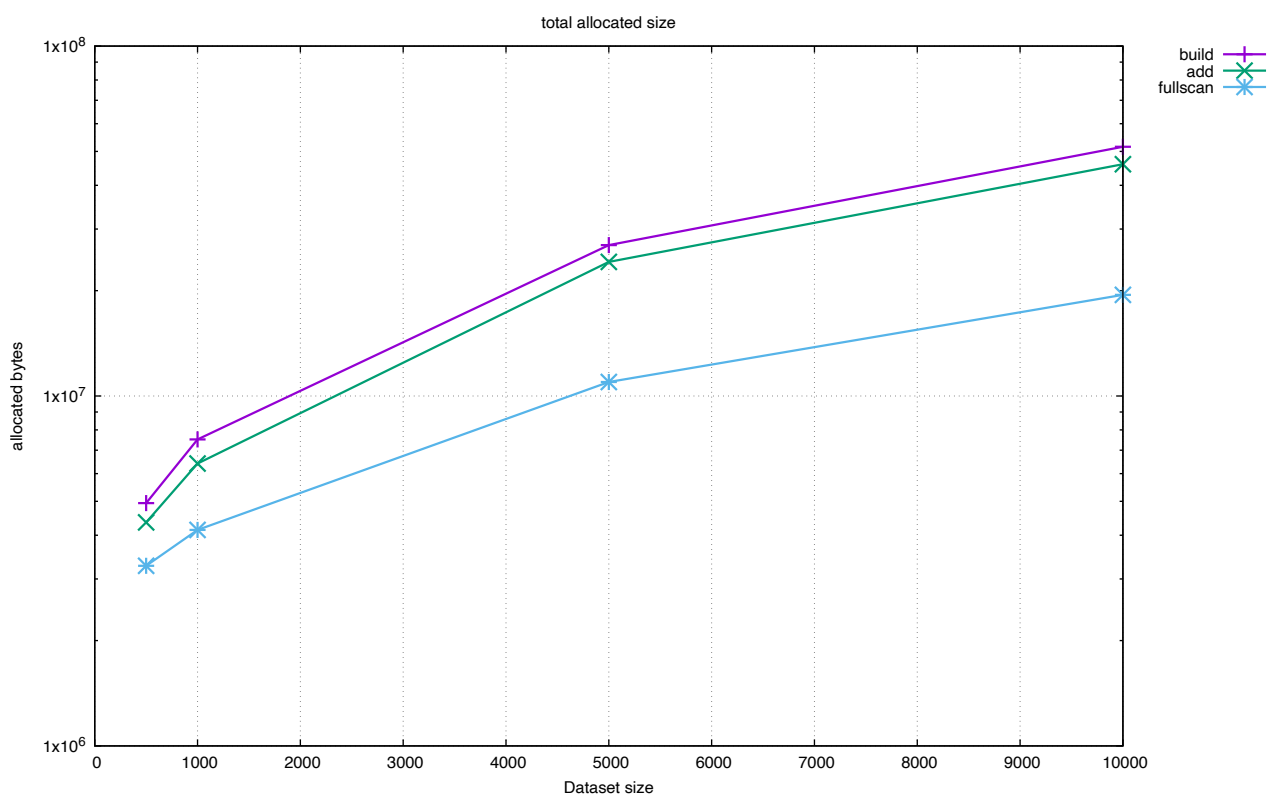


Рисунок 3.8 – Суммарный размер аллокаций для min hash

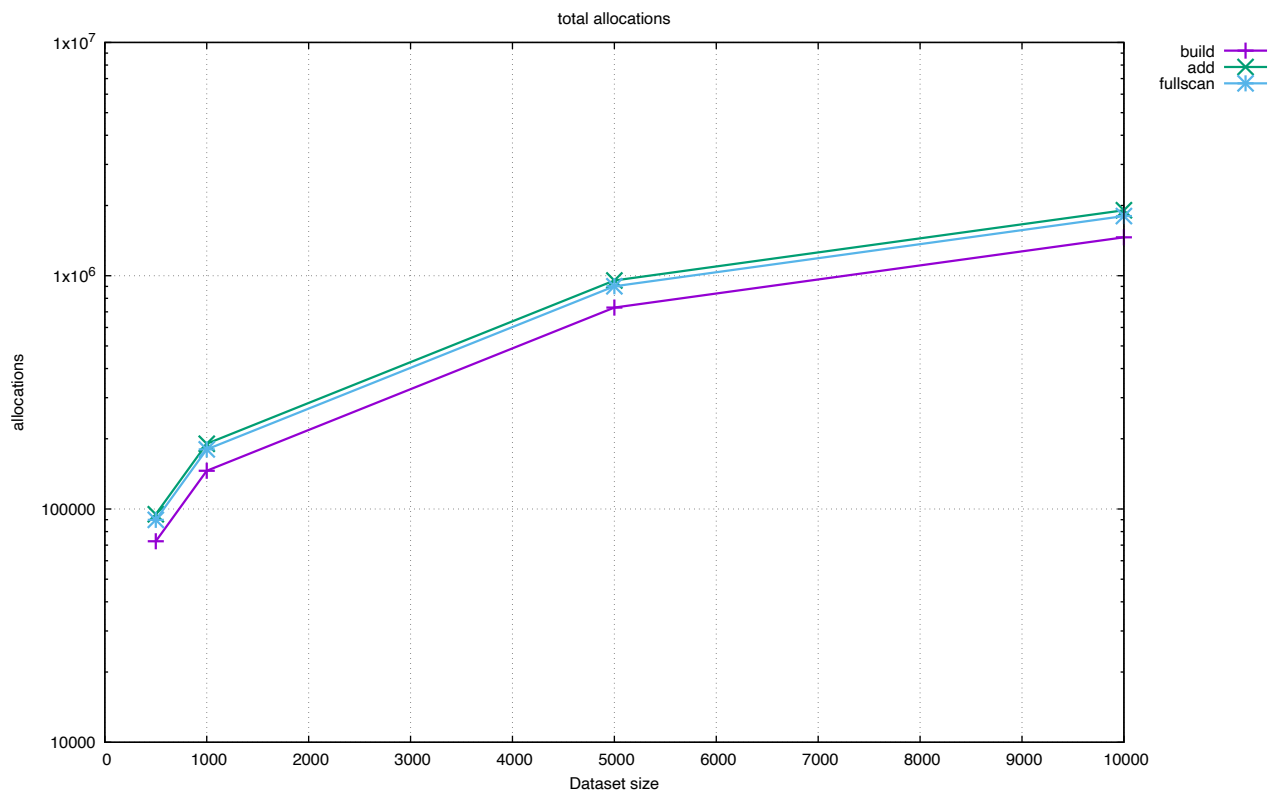


Рисунок 3.9 – Общее число аллокаций для min hash

4 Вывод

Реализована perfect hash таблица для строковых ключей с поддержкой операций вставки и поиска, extendible hash таблица с файловым хранением на базе `mmap`, поддерживающая динамические операции вставки, чтения, обновления и удаления, а также min hash LSH индекс для поиска почти дубликатов в текстовых коллекциях.

Проведены замеры производительности на наборах данных разного размера. Для perfect hashing подтверждена эффективность на статическом множестве ключей, а для extendible hashing показано, что операции **get** и **update** остаются самыми быстрыми, тогда как **insert** и особенно **delete** несут дополнительные издержки на модификацию структуры каталогов и бакетов. Для min hash подтверждена линейная масштабируемость операций **build** и **add** (≈ 16 нс/элемент) при квадратичном росте задержки **fullscan** — базового алгоритма перебора, используемого для эталонного сравнения. Результаты представлены в таблицах и на графиках.